# OP-TEE (Open Portable Trusted Execution Environment) 구현체 분석

## 개요

OP-TEE는 ARM TrustZone 기술을 기반으로 한 오픈소스 Trusted Execution Environment (TEE) 구현체입니다. 이 문서는 OP-TEE OS의 secure side 구현체에 대한 상세한 분석을 제공합니다.

## 프로젝트 구조

### 최상위 디렉토리 구조

```
optee_os/
├── core/ # OP-TEE 커널 및 핵심 컴포넌트
├── ldelf/ # 동적 링커 및 ELF 로더
├── lib/ # 공유 라이브러리들
├── ta/ # Trusted Application 관련 코드
├── mk/ # 빌드 시스템 및 Makefile 규칙
├── scripts/ # 빌드 및 유틸리티 스크립트
├── keys/ # 암호화 키 파일들
└── .devcontainer/ # 개발 환경 설정
```

## 핵심 컴포넌트 분석

# 1. Core 모듈 (core/)

OP-TEE의 핵심 운영체제 기능을 담당하는 모듈입니다.

## 1.1 아키텍처 지원 (core/arch/)

- **ARM 아키텍처**: ARMv7-A, ARMv8-A 지원
- **RISC-V 아키텍처**: RISC-V 64비트 지원
- 플랫폼별 특화 코드 및 설정

## 1.2 암호화 모듈 (core/crypto/)

```
// 주요 암호화 기능들
- AES (Advanced Encryption Standard)
- AES-GCM (Galois/Counter Mode)
- AES-CTS (Ciphertext Stealing)
- AES-CBC, AES-CTR, AES-ECB, AES-XTS
- SM 시리즈 (중국 국가 표준)
- SM2 (공개키 암호화)
- SM3 (해시 함수)
- SM4 (대칭키 암호화)
- 난수 생성기 (RNG)
- Fortuna 알고리즘
- 하드웨어 RNG 지원
```

## 1.3 드라이버 모듈 (core/drivers/)

다양한 하드웨어 플랫폼을 지원하는 드라이버들:

**주요 드라이버 카테고리:**

- **UART 드라이버**: 시리얼 통신 (pl011, ns16550, stm32_uart 등)
- **암호화 하드웨어**: CAAM, TRNG, PUF 등
- **인터럽트 컨트롤러**: GIC, PLIC, APLIC
- **타이머 및 워치독**: 다양한 플랫폼별 타이머
- **GPIO 및 핀 제어**: 범용 입출력 제어
- **전원 관리**: PM, 클럭 제어

### 1.4 커널 모듈 (core/kernel/)

OP-TEE 커널의 핵심 기능들:

**주요 기능:**

- **부팅 프로세스** ( `boot.c` ): 시스템 초기화
- **스레드 관리** ( `thread.c` ): 멀티스레딩 지원
- **뮤텍스 및 동기화** ( `mutex.c` ): 동기화 프리미티브
- **인터럽트 처리** ( `interrupt.c` ): 인터럽트 관리
- **시스템 콜** ( `scall.c` ): 시스템 호출 처리
- **TA 관리** ( `tee_ta_manager.c` ): Trusted Application 관리
- **메모리 관리 인터페이스**: 사용자 모드 컨텍스트 관리

### 1.5 메모리 관리 (core/mm/)

```
// 메모리 관리 주요 컴포넌트
- core_mmu.c // 코어 MMU 관리
- page_alloc.c // 페이지 할당자
- vm.c // 가상 메모리 관리
```

```
- mobj.c // 메모리 객체 관리
- tee_mm.c // TEE 메모리 매니저
```

### 1.6 PTA (Pseudo Trusted Applications) (core/pta/)

시스템 레벨 서비스를 제공하는 내장 TA들:

- **시스템 PTA** ( `system.c` ): 시스템 정보 및 제어
- **암호화 PTA**: 하드웨어 암호화 기능 접근
- **통계 PTA** ( `stats.c` ): 시스템 통계 정보
- **RTC PTA** ( `rtc.c` ): 실시간 클럭 관리
- **디바이스 PTA** ( `device.c` ): 디바이스 관리

### 1.7 TEE 서비스 (core/tee/)

TEE API 구현 및 서비스:

- **암호화 서비스** ( `tee_svc_cryp.c` ): TEE 암호화 API
- **스토리지 서비스** ( `tee_svc_storage.c` ): 보안 스토리지
- **파일 시스템**:
- `tee_ree_fs.c` : REE 파일 시스템 인터페이스
- `tee_rpmb_fs.c` : RPMB 기반 보안 파일 시스템
- **객체 관리** ( `tee_obj.c` , `tee_pobj.c` ): TEE 객체 관리

## 2. 동적 링커 (ldelf/)

OP-TEE의 동적 링커 및 ELF 로더 구현:

```
// 주요 컴포넌트
```

```
- main.c // 링커 메인 로직
- dl.c, dl.h // 동적 링킹 구현
- ta_elf.c // TA ELF 파일 처리
- sys.c // 시스템 인터페이스
- ftrace.c // 함수 추적 기능
```

**기능:**

- ELF 파일 로딩 및 재배치
- 동적 심볼 해석
- 메모리 보호 설정
- 함수 추적 지원

## 3. 라이브러리 (lib/)

공유 라이브러리들:

### 3.1 libutils

- 범용 유틸리티 함수들
- 문자열 처리, 메모리 관리 등

### 3.2 libutee

- User TEE API 구현
- TA에서 사용하는 표준 API

### 3.3 libmbedtls

- mbedTLS 암호화 라이브러리

- TLS/SSL 및 암호화 기능

### 3.4 libunw

- 스택 언와인딩 라이브러리
- 디버깅 및 예외 처리 지원

## 4. Trusted Application 지원 (ta/)

TA 개발 및 실행 환경:

```
// 주요 구성 요소
- ta.mk // TA 빌드 규칙
- link.mk // 링킹 규칙
- user_ta_header.c // TA 헤더 구현
```

**내장 TA들:**

- **PKCS#11 TA**: 암호화 토큰 인터페이스
- **AVB TA**: Android Verified Boot
- **Trusted Keys TA**: 키 관리
- **RemoteProc TA**: 원격 프로세서 관리

# 빌드 시스템

## Makefile 구조

```
# 주요 빌드 변수들
ARCH ?= arm # 타겟 아키텍처
```

```
PLATFORM ?= vexpress # 타겟 플랫폼

O := out # 출력 디렉토리



# 빌드 타겟들

all: # 전체 빌드

clean: # 정리

cscope: # 코드 인덱싱

checkpatch: # 코드 스타일 검사
```

### 설정 시스템

- [conf.mk](): 빌드 설정 파일
- *CFG_ 변수들**: 기능 활성화/비활성화
- **플랫폼별 설정**: `core/arch/*/plat-*/conf.mk`

## 보안 기능

### 1. 메모리 보호

- **MMU 기반 메모리 격리**: Secure/Non-secure 메모리 분리
- **스택 보호**: 스택 오버플로우 방지
- **ASLR**: 주소 공간 레이아웃 랜덤화

### 2. 암호화 지원

- **하드웨어 암호화 엔진 활용**

- **다양한 암호화 알고리즘 지원**
- **키 관리 및 보안 스토리지**

### 3. 무결성 검증

- **TA 서명 검증**
- **부팅 시 무결성 체크**
- **런타임 검증**

## 디버깅 및 개발 도구

### 1. 추적 및 로깅

```
// 로그 레벨 설정
TRACE_LEVEL // 추적 레벨 제어
CFG_TEE_CORE_LOG_LEVEL // 코어 로그 레벨
```

### 2. 개발 도구

- **ftrace**: 함수 호출 추적
- **gprof**: 프로파일링 지원
- **ASAN**: 주소 새니타이저
- **lockdep**: 데드락 검출

### 3. 테스트 프레임워크

- **core/tests/**: 내장 테스트 케이스

- **CFG_TEE_CORE_EMBED_INTERNAL_TESTS**: 테스트 활성화

# 플랫폼 지원

## ARM 플랫폼

- **Versatile Express**: 개발 보드
- **i.MX**: NXP 프로세서
- **STM32**: STMicroelectronics
- **Xilinx Zynq**: FPGA 플랫폼
- **Broadcom**: BCM 시리즈

## RISC-V 플랫폼

- **SiFive**: RISC-V 구현체
- **Generic RISC-V**: 표준 RISC-V

# 성능 최적화

## 1. 메모리 최적화

- **페이지 캐싱**: `pgt_cache.c`
- **동적 공유 메모리**: `mobj_dyn_shm.c`
- **메모리 풀링**: 효율적인 메모리 할당

## 2. 암호화 최적화

- **하드웨어 가속**: 플랫폼별 암호화 엔진 활용
- **어셈블리 최적화**: 아키텍처별 최적화 코드

### 3. 컨텍스트 스위칭 최적화

- **빠른 시스템 콜**: 최적화된 시스템 콜 인터페이스
- **효율적인 스레드 관리**: 컨텍스트 스위칭 오버헤드 최소화

## 확장성 및 모듈화

### 1. 드라이버 프레임워크

- **플러그인 아키텍처**: 새로운 하드웨어 지원 용이
- **디바이스 트리 지원**: 하드웨어 설정 추상화

### 2. 암호화 라이브러리 선택

```
// 지원되는 암호화 라이브러리
CFG_CRYPTOLIB_NAME=tomcrypt // LibTomCrypt
CFG_CRYPTOLIB_NAME=mbedtls // mbedTLS
```

### 3. 설정 기반 빌드

- **기능별 활성화/비활성화**: CFG_* 매크로
- **플랫폼별 커스터마이징**: 유연한 설정 시스템

이 문서는 OP-TEE 구현체의 주요 구조와 기능을 개괄적으로 설명합니다. 각 모듈에 대한 더 자세한 분석이 필요하시면 특정 영역에 대해 추가 문의해 주세요.